

Simulation and Analysis of Customer Queuing System in a Bank

By Hirudika Anupama

2025-04-11

1. Introduction

Since a well-established bank has to deal with a high number of customers, it is necessary to simulate the current queuing system and predict future performance under different scenarios.

The goal is to assess the impact of the current service counter process (with 2 service counters) and add a service counter (3 service counters) if the average wait time is more than 15 minutes.

Customers arrive in a Poisson process (10 per hour) and are served with exponentially distributed times (average service time 5 minutes). The simulation covers an 8-hour workday (480 minutes).

2. Model Development

2.1 The concept of the model

The bank's customer service system follows the M/M/2 queuing model:

1. M (poisson Arrivals) – Customers arrive randomly at a normal rate ($\lambda = 10/\text{hour}$).
2. M (Exponential Service Times) – Service times vary randomly, averaging 5 minutes per customer ($\mu = 12/\text{hour per server}$).
3. 2 Servers – Two counters operate in parallel on a shared queue.

2.2. Key Features

1. First-come, first-served (FCFS) discipline.
2. Unlimited queue capacity (no customer rejection).
3. Markovian (memoryless) property: Past arrivals/service times do not affect future events.

2.3 Model Formulation

Arrival Process

Customers arrive randomly according to a Poisson process (random independent events occur at a constant rate). There are 10 customers per hour, and the arrival rate per minute is:

$$Rate = \frac{10}{60} \text{ customers per minute.}$$

Service Process

Service times follow an exponential distribution (time between random independent events) with a mean of 5 minutes. This means that the service rate is:

$$Rate = \frac{1}{5} \text{ per minute.}$$

Queue Discipline

Customers are served on a first-come, first-served (FCFS) basis.

Simulation Duration

The simulation runs for 480 minutes (8 hours).

2.4 Implementation Using the “simmer” Package

The simmer package, designed for discrete-event simulation, is used for this. In the simulation, each customer travels through the following trajectory:

1. Seize a server: The customer occupies a server.
2. Wait (if necessary) and get served: The customer service time is generated from an exponential distribution.
3. Release the server: After service, the server becomes available for the next customer.

Load Libraries

```
# Required libraries for discrete-event simulation and visualization  
library(simmer) # Core simulation engine  
  
## Warning: package 'simmer' was built under R version 4.3.3
```



```

    timeout(function() rexp(1, customer_service_rate)) %>%           # Get
    served for a time sampled from an exponential distribution
    release("teller_counter")                                       # Release
    the server

```

Simulate for 2 Servers (Current Scenario)

Initialize the Simulation Environment

This creates a bank simulation with two servers, generates customers arriving at exponentially distributed intervals, and simulates a specified time (480 minutes).

```

# Create simulation environment
bank_sim_two_counters <- simmer("Bank with 2 Tellers")

# Add 2 servers (resources)
bank_sim_two_counters <- bank_sim_two_counters %>%
  add_resource("teller_counter", capacity = 2)

# Generate customers with exponential inter arrival time
bank_sim_two_counters <- bank_sim_two_counters %>%
  add_generator("customer", customer_trajectory, function() rexp(1,
    customer_arrival_rate))

# Simulate for 480 minutes
bank_sim_two_counters <- bank_sim_two_counters %>%
  run(until = bank_open_duration)

```

Extract Monitoring Data

Gathers time-stamped records of customer arrivals/departures and server status changes during the simulation.

```

# Extract data about customer arrivals and server usage
arrival_log_two_counters <- get_mon_arrivals(bank_sim_two_counters)
resource_log_two_counters <- get_mon_resources(bank_sim_two_counters)

```

Calculate Waiting Time

Queue time is calculated by subtracting the service time from each customer's total system time.

```

# Calculate waiting time for each customer
arrival_log_two_counters$waiting_time <- arrival_log_two_counters$end_time -
  arrival_log_two_counters$start_time
-

```

```
arrival_log_two_counters$activity_time
```

Calculate performance metrics for 2 tellers

```
# Average waiting time
avg_wait_time_two_counters <- mean(arrival_log_two_counters$waiting_time)

# Average queue length over time
avg_queue_length_two <- mean(resource_log_two_counters$queue)

# Server utilization = total busy time / total available time (all servers)
server_utilization_two <- sum(resource_log_two_counters$server) /
(bank_open_duration * 2)

# Display results
cat("----- 2 SERVICE TELLERS -----\\n")

## ----- 2 SERVICE TELLERS -----

cat("Average Wait Time:", round(avg_wait_time_two_counters, 2), "minutes\\n")

## Average Wait Time: 0.17 minutes

cat("Average Queue Length:", round(avg_queue_length_two, 2), "customers\\n")

## Average Queue Length: 0.09 customers

cat("Server Utilization:", round(server_utilization_two * 100, 2), "%\\n\\n")

## Server Utilization: 15.31 %
```

Simulate for 3 Servers (Proposed Scenario)

Similarly, we set up a simulation for the 3-server scenario by changing the resource capacity from 2 to 3. The same steps are followed to run the simulation and extract monitoring data

Initialize the Simulation Environment

```
# Create simulation environment
bank_sim_three_counters <- simmer("Bank with 3 Tellers")

# Add 2 servers (resources)
bank_sim_three_counters <- bank_sim_three_counters %>%
  add_resource("teller_counter", capacity = 3)

# Generate customers with exponential inter arrival time
bank_sim_three_counters <- bank_sim_three_counters %>%
```

```
add_generator("customer", customer_trajectory, function() rexp(1,
customer_arrival_rate))
```

```
# Simulate for 480 minutes
```

```
bank_sim_three_counters <- bank_sim_three_counters %>%
  run(until = bank_open_duration)
```

Extract Monitoring Data

```
# Get arrival and resource logs
```

```
arrival_log_three_counters <- get_mon_arrivals(bank_sim_three_counters)
resource_log_three_counters <- get_mon_resources(bank_sim_three_counters)
```

Calculate Waiting Time

```
# Calculate waiting time for each customer
```

```
arrival_log_three_counters$waiting_time <-
arrival_log_three_counters$end_time -
```

```
arrival_log_three_counters$start_time -
```

```
arrival_log_three_counters$activity_time
```

Compute Metrics for 3 Servers

```
# Calculate performance metrics for 3 tellers
```

```
# Average waiting time
```

```
avg_wait_time_three_counters <- mean(arrival_log_three_counters$waiting_time)
```

```
# Average queue length over time
```

```
avg_queue_length_three <- mean(resource_log_three_counters$queue)
```

```
# Server utilization = total busy time / total available time (all servers)
```

```
server_utilization_three <- sum(resource_log_three_counters$server) /
(bank_open_duration * 3)
```

```
cat("----- 3 SERVICE TELLERS -----\\n")
```

```
## ----- 3 SERVICE TELLERS -----
```

```
cat("Average Wait Time:", round(avg_wait_time_three_counters, 2),
"minutes\\n")
```

```
## Average Wait Time: 0.13 minutes
```

```
cat("Average Queue Length:", round(avg_queue_length_three, 2), "customers\\n")
```

```
## Average Queue Length: 0.05 customers
```

```
cat("Server Utilization:", round(server_utilization_three * 100, 2), "%\\n")
```

```
## Server Utilization: 12.92 %
```

3. Analysis and Interpretation

3.1 Overview

The bank’s queuing system was simulated using the simmer package in R. Two scenarios were modeled: one with 2 tellers and one with 3 tellers. The aim was to evaluate how increasing the number of service counters would affect key performance metrics such as average waiting time, queue length, and server utilization.

3.2 Simulation Results

The simulation produced the following results:

Queue Simulation Results Comparison		
Metric	2 Tellers	3 Tellers
Average Waiting Time (minutes)	0.17	0.13
Average Queue Length (customers)	0.09	0.05
Server Utilization (%)	15.31	12.92

3.3 Interpretation of Metrics

Waiting Time:

With 2 tellers, the average wait time is 0.17 minutes (approximately 10.2 seconds).

With 3 tellers, the average wait time is 0.13 minutes (approximately 7.8 seconds).

Both cases show very low waiting times, which shows that the system is very efficient even with 2 tellers. Adding a third teller reduces the waiting time slightly.

Queue Length:

The average queue length with 2 tellers is about 0.09 customers, and when a third teller is added, it drops to 0.05 customers.

Due to the very short queue times, customers are treated almost immediately. With 3 tellers, the reduction in queue times is minimal, further emphasizing the current system’s efficiency.

Server Utilization:

In the 2-teller setup, the servers are utilized at 15.31%.

In the 3-teller setup, the utilization drops to 12.92%.

The low server utilization in both cases indicates that the tellers are not heavily loaded and there is significant downtime. Adding a third teller reduces the utilization slightly, and the low overall values suggest that the system is not overloaded.

3.4 Operational Implications

The current system works very well. With 2 tellers, customers wait about 10 seconds, and with 3 tellers, the wait is almost nothing. The system is not crowded, and tellers are not too busy.

Adding a third teller would make things slightly faster, but it may not be worth the extra cost because the system already runs smoothly with short wait times.

3.5 Recommendation

The simulation indicates that the current two-teller system operates very effectively, with customers experiencing an average wait of only about 10 seconds. Adding a third teller would slightly reduce waiting times, but the improvement is so minimal that it may not justify the additional cost. Since the system isn't overloaded and the tellers aren't excessively busy, the bank should maintain its two-teller setup for now. If customer traffic increases in the future, they can reassess this decision, but for now, an expansion isn't necessary.

4. Visualization

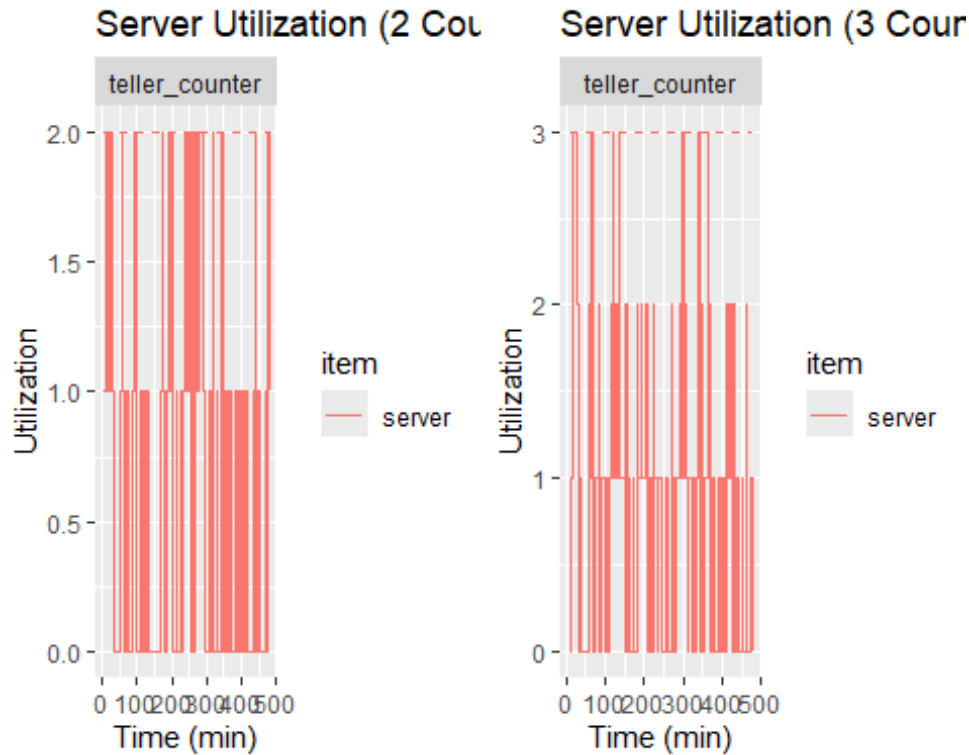
The following visualizations illustrate the effect of increasing the number of service counters in a bank's queuing system from two to three. The line graphs indicate that server utilization and customer wait times are more stable and significantly lower with three counters, while the histograms reveal that most customers are served almost immediately. Overall, adding a third counter enhances efficiency and reduces queue congestion.

4.1 Server Utilization Over Time

```
#Server Utilization Over Time (2 Counters)
server_utilizatio_two <- plot(resource_log_two_counters,
  metric = "usage",
  items = "server",
  step = TRUE) +
  ggtitle("Server Utilization (2 Counters)") +
  xlab("Time (min)") +
  ylab("Utilization")

# Server Utilization Over Time (3 Counters)
server_utilizatio_three <- plot(resource_log_three_counters,
  metric = "usage",
  items = "server",
  step = TRUE) +
  ggtitle("Server Utilization (3 Counters)") +
  xlab("Time (min)") +
  ylab("Utilization")

grid.arrange(server_utilizatio_two, server_utilizatio_three, ncol = 2, widths
= c(30,30))
```



The 1st graph shows how much of the total server capacity (Teller 2) is used throughout the 8 hours. The red lines show frequent switching between idle and busy states. According to the graph, the second teller is not used as much as the first one. This means the system could handle more customers to make better use of both counters.

2nd graph illustrates the total server capacity (Teller 3) utilized over the 8 hours. According to the graph, the second counter is used less frequently than the first, while the third counter sees minimal usage. Given the current customer arrival rate, adding a third counter leads to low utilization and results in greater resource wastage compared to the two-counter setup.

4.2 Customer Waiting Times

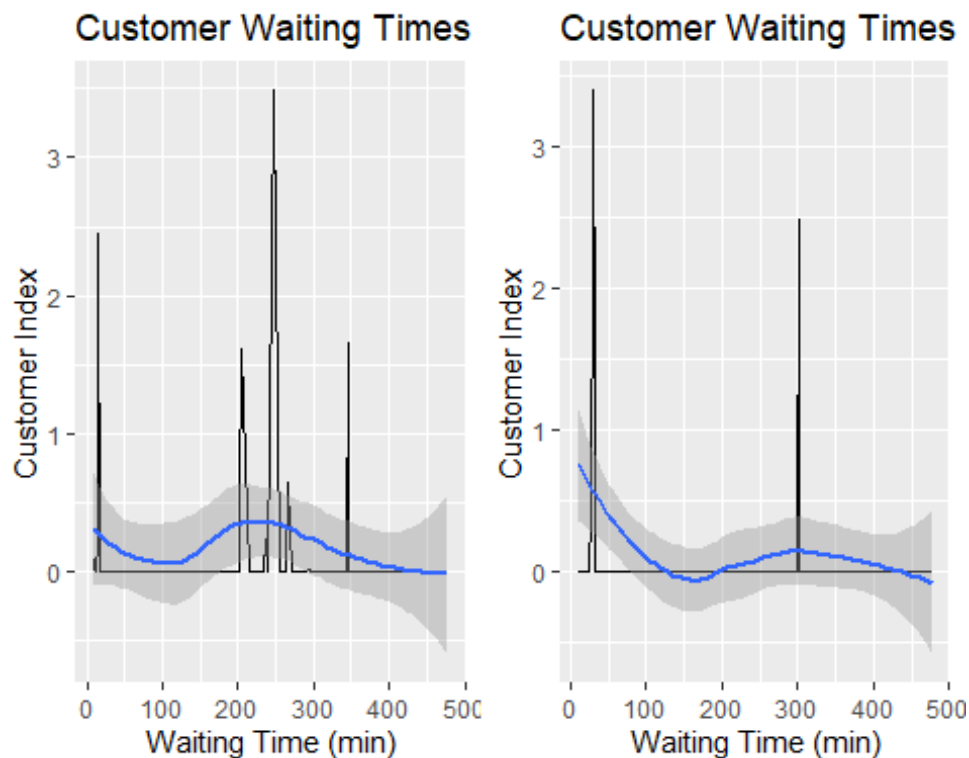
```
# Customer Waiting Times (2 Counters)
waiting_times_two <- plot(arrival_log_two_counters,
  metric = "waiting_time") +
  ggtitle("Customer Waiting Times (2 Counters)") +
  xlab("Waiting Time (min)") +
  ylab("Customer Index")

# Customer Waiting Times (3 Counters)
waiting_times_three <- plot(arrival_log_three_counters,
  metric = "waiting_time") +
  ggtitle("Customer Waiting Times (3 Counters)") +
```

```
xlab("Waiting Time (min)") +
ylab("Customer Index")
```

```
grid.arrange(waiting_times_two, waiting_times_three, ncol=2)
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



Teller with 2 Counters: The waiting times are more variable and exhibit several sharp spikes, indicating occasional congestion and longer delays for some customers.

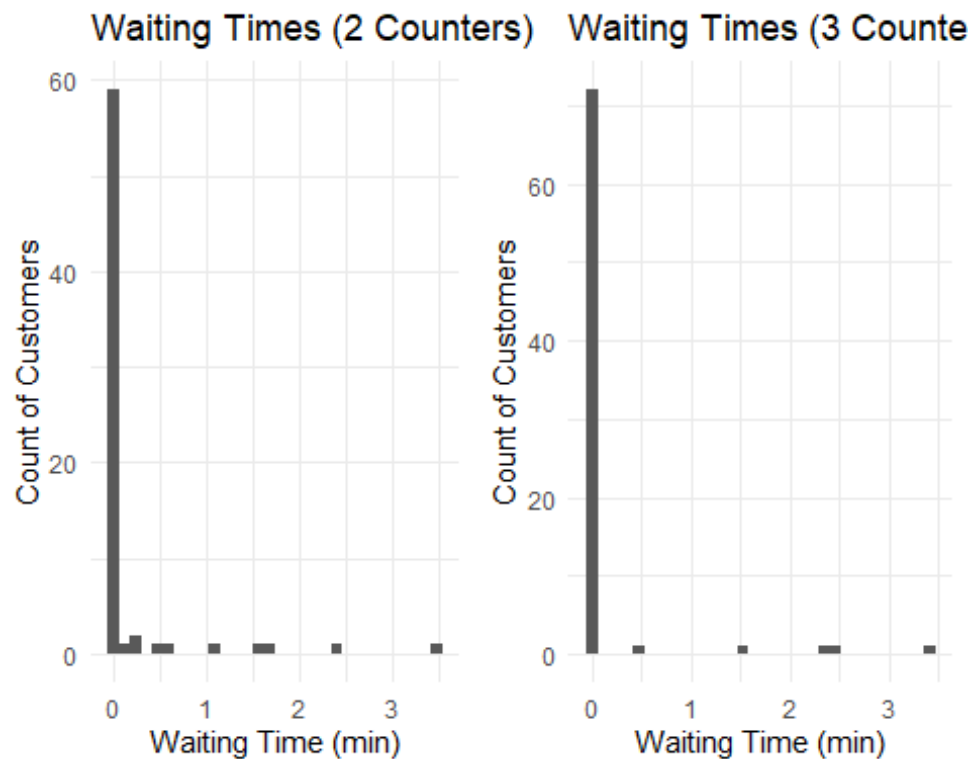
Teller with 3 Counters: Waiting times are more consistently low and decrease more rapidly over time, suggesting smoother and more efficient service delivery.

4.3 Customer Waiting Times Histogram

```
# Histogram of Waiting Times (2 Counters)
waiting_times_two_hist <- ggplot(arrival_log_two_counters, aes(x =
waiting_time)) +
  geom_histogram(bins = 30) +
  ggtitle("Waiting Times (2 Counters)") +
  xlab("Waiting Time (min)") +
  ylab("Count of Customers") +
  theme_minimal()
```

```
# Customer Waiting Times (3 Counters)
waiting_times_three_hist <- ggplot(arrival_log_three_counters, aes(x =
waiting_time)) +
  geom_histogram(bins = 30) +
  ggtitle("Waiting Times (3 Counters)") +
  xlab("Waiting Time (min)") +
  ylab("Count of Customers") +
  theme_minimal()

grid.arrange(waiting_times_two_hist, waiting_times_three_hist, ncol=2, widths
= c(30, 31))
```



2 Counters: Most customers experience nearly zero wait times, but there are more instances of higher wait times, indicating occasional congestion.

3 Counters: Almost all customers are served with minimal delay, and higher wait times are even rarer, demonstrating significantly improved efficiency.